MASTER EN INGENIERÍA Y DESARROLLO DE SOLUCIONES CON INTELIGENCIA ARTIFICIAL GENERATIVA

TRABAJO FINAL DE MÁSTER

ContratIA

Asistente inteligente para la generación, revisión y consulta de contratos en derecho español

Autor: Adrián Vizcaíno Galán
Repositorio: https://github.com/adrianvizga/EBIS_TFM_Contratia

Julio de 2026

Resumen

ContratIA es una aplicación web que aplica modelos de lenguaje de gran tamaño (LLMs) y técnicas de recuperación aumentada de información (RAG) para resolver un problema cotidiano de muchas personas y empresas: entender, generar y consultar contratos en el contexto del derecho español.

La plataforma ofrece tres funcionalidades principales. La primera es el análisis de contratos, que permite al usuario cargar un documento en formato PDF, DOCX o TXT y recibir un desglose cláusula a cláusula con identificación de riesgos, aspectos positivos y referencias a la legislación aplicable. La segunda es un chatbot legal, que responde preguntas sobre contratos y legislación española apoyándose en un corpus jurídico indexado en una base de datos vectorial. La tercera es la generación de contratos, que produce borradores de contratos tipo (arrendamiento de vivienda, arrendamiento de local y prestación de servicios) a partir de los datos que proporciona el usuario.

Técnicamente, el sistema combina FastAPI como backend, React como frontend, PostgreSQL para la persistencia relacional, Qdrant como base de datos vectorial, Redis como cola de tareas y Celery para la ejecución asíncrona de análisis. Los modelos de OpenAI (GPT 5.4-mini y text-embedding-3-small) son el motor de inteligencia artificial. Todo el stack se despliega mediante Docker Compose.

El corpus jurídico RAG está formado 2.786 fragmentos que representan textos legales oficiales obtenidos del BOE (Boletín Oficial del Estado), incluyendo el Código Civil, la Ley de Arrendamientos Urbanos, el RGPD, la LOPDGDD y el Código de Comercio.

ÍNDICE DE CONTENIDO

Resumen **2**

1. Introducción **5**

1.1. Contexto y motivación 5

1.2. Objetivos generales y específicos 6

1.3. Alcance de la solución 6

2. Estado del arte y fundamentos teóricos **8**

2.1. Modelos de lenguaje y arquitectura Transformer 8

2.2. Técnicas de adaptación: prompting, RAG y fine-tuning 9

2.3. Marco legal y ético 10

3. Requisitos y diseño de la solución **12**

3.1. Requisitos funcionales y no funcionales 12

3.2. Arquitectura de la aplicación 13

3.3. Selección tecnológica 14

3.4. Diseño de datos y pipelines 15

4. Implementación **18**

4.1. Estructura del código y componentes 18

4.2. Integración con modelos OpenAI 21

4.3. Gestión de prompts y orquestación 22

4.4. Mecanismos de seguridad 24

4.5. Despliegue e infraestructura 26

5. Evaluación y experimentos realizados **28**

5.1. Métricas de evaluación 28

5.2. Resultados cuantitativos y cualitativos 29

5.3. Análisis de errores y limitaciones 30

6. Pruebas y validación de producto **32**

6.1. Tests unitarios y funcionales 32

6.2. UX y usabilidad 33
6.3. Observabilidad en producción 34
7. Discusión 35
7.1. Lecciones aprendidas durante el desarrollo 35
7.2. Riesgos, ética y mitigaciones 36
8. Conclusiones y trabajo futuro 38
9. Bibliografía 40
Anexo A. Seguridad en profundidad 42
A.1. Flujo de autenticación 42
A.2. Validación de entradas en la cadena RAG 43
Anexo B. Pipeline RAG: detalle técnico 45
B.1. Proceso de normalización de textos legales 45
B.2. Estrategia de chunking 46
B.3. Generación de embeddings 47
Anexo C. Evolución del proyecto durante el desarrollo 49
C.1. De la idea inicial al prototipo funcional 49
C.2. Decisiones técnicas que cambiarían en retrospectiva 51

1. Introducción

1.1. Contexto y motivación

Los contratos son documentos fundamentales en la vida cotidiana de personas y empresas. Desde el alquiler de una vivienda hasta un acuerdo de prestación de servicios entre dos empresas, los contratos fijan derechos y obligaciones con consecuencias económicas y legales que pueden ser muy relevantes. Sin embargo, la comprensión de estos textos requiere conocimientos jurídicos que la mayoría de la población no tiene, lo que provoca que muchas personas los firmen sin entender completamente sus implicaciones.

Acudir a un abogado para que revise un contrato es la opción más segura, pero también la más cara y la menos accesible para particulares o para pequeñas empresas que firman decenas de contratos al año. Esta brecha entre la necesidad de asesoramiento legal y el acceso real a él es el punto de partida de ContratIA.

La irrupción de los modelos de lenguaje de gran escala (LLMs) en los últimos años ha abierto una posibilidad real: construir herramientas capaces de leer, analizar y explicar textos legales con un nivel de comprensión suficiente para ser útiles a un usuario sin formación jurídica. Modelos como GPT-5 o la familia GPT-4o de OpenAI han demostrado resultados muy competentes en tareas de comprensión lectora, extracción de información y razonamiento sobre textos especializados.

A esto hay que sumarle el avance en técnicas como la Recuperación Aumentada de Información (RAG, por sus siglas en inglés), que permite combinar la capacidad generativa de los LLMs con una base de conocimiento actualizable. En el ámbito legal esto es especialmente valioso porque las leyes cambian y una respuesta basada en el texto vigente oficial tiene mucho más valor que una generada solo desde el conocimiento del modelo, que puede estar desactualizado o ser impreciso.

El proyecto ContratIA nació de la convicción de que es posible construir una herramienta que democratice el acceso al análisis legal de contratos (sin pretender sustituir al abogado en casos complejos) para ayudar a que el usuario entienda qué está firmando y pueda tomar decisiones informadas.

El contexto del Máster en Ingeniería y Desarrollo de Soluciones con Inteligencia Artificial Generativa ha sido el marco ideal para abordar este reto, ya que permite aplicar de forma integrada los conocimientos adquiridos en modelos de lenguaje,

diseño de sistemas RAG, ingeniería de prompts, desarrollo de APIs, infraestructura con contenedores y principios de diseño de producto.

1.2. Objetivos generales y específicos

El objetivo general del proyecto es desarrollar una aplicación web funcional que, apoyándose en modelos de lenguaje y en un corpus jurídico oficial, permita a usuarios no expertos analizar, consultar y generar contratos en el marco del derecho español.

Los objetivos específicos son los siguientes:

- Diseñar e implementar un sistema de análisis de contratos cláusula a cláusula, capaz de identificar riesgos, señalar aspectos positivos o negativos y citar los artículos legales aplicables.
- Construir un chatbot legal con memoria conversacional que responda preguntas sobre contratos y legislación española, citando siempre las fuentes del corpus.
- Desarrollar un módulo de generación de contratos tipo que produzca borradores estructurados para los tipos de contrato más habituales.
- Construir un corpus jurídico RAG a partir de textos legales oficiales del BOE.
- Desplegar la aplicación completa mediante Docker Compose.

1.3. Alcance de la solución

ContratIA está diseñado para cubrir los contratos más habituales en el marco del derecho civil y mercantil español. Los tipos de contrato que el sistema puede analizar, consultar y generar son los siguientes:

- Contratos de arrendamiento de vivienda.
- Contratos de arrendamiento de local de negocio.
- Contratos de prestación de servicios entre empresas o entre empresa y autónomo.

Fuera del alcance del proyecto quedan, por razones de complejidad y de falta de corpus legal específico:

- Contratos de contratación pública o administrativa.
- Contratos internacionales o con elementos de derecho extranjero.
- Contratos sectoriales específicos (banca, seguros, salud, telecomunicaciones, transporte, turismo, energía…).

La aplicación no pretende sustituir el asesoramiento jurídico profesional. Sus respuestas son orientativas e informativas. El sistema incluye advertencias explícitas en la interfaz cuando la complejidad del caso aconseja consultar a un abogado.

2. Estado del arte y fundamentos teóricos

2.1. Modelos de lenguaje y arquitectura Transformer

Los modelos de lenguaje modernos tienen su origen en la arquitectura Transformer, presentada en 2017 por un equipo de investigadores de Google, con Ashish Vaswani como uno de sus principales autores, en el artículo seminal "Attention Is All You Need". Esta arquitectura introdujo el mecanismo de atención multicabeza (multi-head attention) como piedra angular del procesamiento de texto, permitiendo que el modelo capture dependencias entre palabras distantes en una secuencia de forma mucho más eficiente que las redes recurrentes anteriores.

El Transformer se compone de dos bloques principales: el codificador (encoder) y el decodificador (decoder). Dependiendo del caso de uso, los modelos actuales pueden utilizar una de estas partes o combinar ambas. Los modelos tipo GPT (Generative Pre-trained Transformer) de OpenAI son únicamente decodificadores, entrenados para predecir el siguiente token de una secuencia dada; esto los hace especialmente efectivos para tareas generativas como responder preguntas, redactar textos o analizar documentos.

El entrenamiento de estos modelos se realiza en dos fases. La primera es el preentrenamiento, donde el modelo aprende representaciones generales del lenguaje a partir de enormes corpus de texto. La segunda es el fine-tuning, donde el modelo se especializa en tareas concretas. Técnicas como el Aprendizaje por Refuerzo con Retroalimentación Humana (RLHF) han sido clave para que modelos como GPT sean más útiles, seguros y alineados con las instrucciones del usuario.

En ContratIA se utilizan modelos de la familia GPT de OpenAI a través de su API. El modelo principal es GPT en su variante más eficiente disponible (configurado como gpt-5.4-mini en este momento), con un modelo de respaldo (gpt-4.1-mini) que se activa automáticamente ante cualquier fallo del modelo principal en todas las operaciones. Para la generación de embeddings se utiliza text-embedding-3-small, que produce vectores de 1.536 dimensiones con excelente rendimiento en recuperación semántica.

En el ámbito del procesamiento de lenguaje legal, la investigación ha avanzado significativamente en los últimos años. Proyectos como LegalBERT, CamemBERT o modelos específicos para el derecho español han demostrado que el fine-tuning con

corpus jurídicos mejora el rendimiento en tareas como clasificación de cláusulas, extracción de entidades legales o detección de riesgos. Sin embargo, estos modelos requieren recursos computacionales elevados para su entrenamiento y despliegue, lo que hace que para un proyecto de este tipo el uso de la API de OpenAI sea la alternativa más pragmática y con mejor relación coste/calidad.

2.2. Técnicas de adaptación: prompting, RAG y fine-tuning

Una vez que existen modelos de lenguaje de propósito general con alta capacidad, la cuestión es cómo adaptarlos a un dominio específico sin incurrir en los costes del entrenamiento completo. Las tres técnicas principales son el prompting, el RAG y el fine-tuning.

Prompting

El prompting consiste en diseñar la instrucción de entrada al modelo de forma que guíe su comportamiento hacia la respuesta deseada. Las técnicas de prompting han evolucionado desde el simple "zero-shot prompting" (pedir directamente sin ejemplos) hasta el "chain-of-thought prompting" (pedir al modelo que razone paso a paso) o el "few-shot prompting" (incluir ejemplos de entrada-salida en el prompt). En ContratIA se utiliza prompting estructurado con roles claros (system/user), instrucciones precisas de formato JSON de salida y ejemplos de comportamiento esperado. Los prompts están versionados para garantizar la trazabilidad.

Retrieval-Augmented Generation (RAG)

El RAG es una técnica que combina la recuperación de información relevante de una base de conocimiento externa con la capacidad generativa del LLM. El flujo general es:

1. El usuario realiza una consulta
2. El sistema la convierte en un vector de embedding y lo compara contra una base de datos vectorial
3. Se recuperan los fragmentos más relevantes
4. Estos se incluyen en el contexto del prompt
5. El modelo genera una respuesta basada en esa información.

Las ventajas del RAG sobre el fine-tuning para este caso de uso son claras: permite actualizar el conocimiento del sistema sin reentrenar el modelo, garantiza que las respuestas pueden ser trazadas hasta una fuente concreta, y reduce las alucinaciones al anclar la generación en texto real. En ContratIA, el corpus jurídico se indexa en Qdrant y se recupera mediante similitud coseno con el embedding de la consulta.

Fine-tuning

El fine-tuning (y sus variantes eficientes como LoRA o QLoRA) consiste en continuar el entrenamiento de un modelo preentrenado con un conjunto de datos específico del dominio. Esto puede mejorar significativamente el rendimiento en tareas muy concretas, pero requiere datos de entrenamiento de calidad, recursos de cómputo y un proceso de evaluación cuidadoso. Para ContratIA se descartó el fine-tuning por varias razones: la dificultad de obtener suficientes pares de pregunta-respuesta legales etiquetados de calidad, el coste de la infraestructura de entrenamiento y el hecho de que el RAG con modelos GPT ya ofrecía resultados satisfactorios.

2.3. Marco legal y ético

El desarrollo de una aplicación que procesa documentos legales y proporciona orientación jurídica entraña consideraciones éticas y legales que no pueden ignorarse.

Protección de datos personales

Los contratos pueden contener datos personales de las partes firmantes (nombre, DNI, dirección, datos bancarios, etc.). Esto obliga a que la aplicación cumpla con el Reglamento General de Protección de Datos (RGPD) y la Ley Orgánica de Protección de Datos Personales y Garantía de los Derechos Digitales (LOPDGDD). En ContratIA, los documentos subidos se almacenan cifrados en un volumen Docker de acceso controlado. El acceso a los datos está restringido al propietario del contrato mediante control de sesión con JWT. Los usuarios deben aceptar los términos y condiciones, que incluyen la política de privacidad, antes de poder utilizar la aplicación.

Responsabilidad y limitación de uso

Una aplicación que analiza contratos y proporciona opiniones sobre su contenido podría interpretarse como un servicio de asesoramiento jurídico, lo que en España requiere estar colegiado como abogado. Para evitar esta ambigüedad, ContratIA se presenta explícitamente como una herramienta informativa, no como sustituto del consejo profesional. Todas las respuestas incluyen advertencias cuando el caso es suficientemente complejo como para requerir la opinión de un abogado.

Sesgos en los modelos de lenguaje

Los LLMs pueden reflejar sesgos presentes en sus datos de entrenamiento. En el dominio legal, esto podría manifestarse como interpretaciones que favorezcan consistentemente a una de las partes del contrato. Para mitigar este riesgo, el sistema ancla sus análisis en el texto del corpus legal oficial (RAG) y los prompts instruyen al modelo para ser equilibrado y citar siempre la fuente que sustenta cada afirmación.

Propiedad intelectual del corpus

Los textos legales utilizados en el corpus RAG son obtenidos del BOE, cuyo contenido es de libre reutilización para fines no comerciales bajo la licencia de reutilización estándar del BOE. La aplicación respeta esta licencia y mantiene en el manifiesto del corpus los metadatos de origen (identificador BOE, URL, fecha de obtención y hash SHA-256 del contenido) para garantizar la trazabilidad.

3. Requisitos y diseño de la solución

3.1. Requisitos funcionales y no funcionales

Requisitos funcionales

ID	Descripción
RF-01	El usuario puede registrarse con email/contraseña o con su cuenta de Google.
RF-02	El usuario puede subir contratos en formato PDF, DOCX o TXT (máx. 10 MB, 200 páginas).
RF-03	El sistema analiza el contrato cláusula a cláusula e identifica riesgos, aspectos positivos o negativos y referencias legales.
RF-04	El usuario puede consultar el resultado del análisis con detalles por cláusula.
RF-05	El usuario puede exportar el informe de análisis como PDF.
RF-06	El usuario puede iniciar conversaciones con el chatbot legal.
RF-07	El chatbot puede responder con el contrato del usuario como contexto adicional.
RF-08	El chatbot cita las fuentes legales en las que basa su respuesta.
RF-09	El usuario puede generar contratos tipo seleccionando el tipo y completando un formulario.
RF-10	El usuario puede editar los contratos generados y guardar las modificaciones.
RF-11	El usuario puede ver un grafo de relaciones entre cláusulas del contrato analizado.
RF-12	El sistema controla las cuotas de uso según el plan del usuario.

Requisitos no funcionales

ID	Descripción
RNF-01	El análisis de contratos se ejecuta de forma asíncrona para no bloquear la interfaz.
RNF-02	El tiempo de respuesta del chatbot no debe superar los 45 segundos (timeout configurable).
RNF-03	Las contraseñas se almacenan con hash bcrypt; nunca en texto plano.
RNF-04	Los tokens JWT tienen una validez de 24 h (acceso) y 7 días (refresco).
RNF-05	La aplicación incluye cabeceras de seguridad HTTP
RNF-06	Los prompts incluyen protección frente a prompt injection en el system prompt.
RNF-07	El corpus RAG supera los umbrales definidos en el dataset de evaluación antes de desplegarse.
RNF-08	Todos los servicios están contenedorizados con Docker y son reproducibles.
RNF-09	El score de recuperación RAG mínimo aceptable es 0,35 (similitud coseno).
RNF-10	Los logs de cada turno de chat se registran de forma atómica.

3.2. Arquitectura de la aplicación

ContratIA sigue una arquitectura cliente-servidor modular, desplegada mediante Docker Compose en varios servicios especializados. La siguiente descripción recorre los componentes del sistema y sus interacciones.

El sistema se compone de seis servicios principales y dos servicios auxiliares. Los servicios principales son:

- Frontend (React + Nginx): interfaz de usuario servida como SPA (Single Page Application). Nginx actúa como servidor de estáticos y proxy inverso hacia el backend.
- Backend (FastAPI): API REST que implementa toda la lógica de negocio. Gestiona autenticación, análisis de contratos, chatbot, generación y cuotas.
- Worker (Celery): proceso que ejecuta en background las tareas de análisis de contratos, que pueden tardar varios minutos dependiendo de la longitud del documento.
- PostgreSQL: base de datos relacional que almacena usuarios, sesiones, contratos, análisis, cláusulas, riesgos, conversaciones y contratos generados.
- Redis: broker de mensajes para la cola de tareas de Celery.
- Qdrant: base de datos vectorial que almacena los embeddings del corpus jurídico y permite la recuperación semántica.

Además, se cuentan con dos servicios auxiliares para la correcta inicialización de la aplicación:

- db-init: servicio de inicialización que carga el esquema de base de datos al arranque y gestiona migraciones versionadas.
- qdrant-init: servicio que carga el corpus jurídico en Qdrant al arranque si la colección no existe.

El flujo de análisis de un contrato ilustra cómo interactúan estos componentes. El usuario sube el fichero a través del frontend. El backend lo valida, lo almacena en

el sistema de ficheros y crea un registro en PostgreSQL con estado «procesando». A continuación, envía una tarea a la cola Redis. El worker Celery recoge la tarea, extrae el texto del documento, lo divide en cláusulas, llama a la API de OpenAI para analizar cada una (apoyándose en el RAG de Qdrant para obtener contexto legal), almacena los resultados en PostgreSQL y envía una notificación al usuario.

3.3. Selección tecnológica

La selección de tecnologías se basó en dos criterios principales: madurez y estabilidad, y comunidad y documentación.

Componente	Tecnología elegida	Alternativas consideradas	Justificación
Backend API	FastAPI (Python 3.11)	Django, Flask, Express.js	Validación de datos con Pydantic, rendimiento asíncrono, generación automática de OpenAPI.
Frontend	React 19 + Vite + Tailwind CSS	Vue.js, Angular, Next.js	Ecosistema maduro, componentes reutilizables. Vite por velocidad de build. TailWind por estar enfocado a Utilily-first sin css personalizado.
Base de datos relacional	PostgreSQL 15	MySQL, SQLite	Robustez y soporte de JSONB.
Base de datos vectorial	Qdrant 1.13	Pinecone, Weaviate, pgvector	Open source, desplegable on-premise, filtros avanzados.
Cola de tareas	Celery + Redis 7	RQ, Dramatiq	Estándar en Python, soporte de reintentos, integración con backend.
Modelos de IA	OpenAI GPT 5.4-mini + text-embedding-3-small	Claude, Gemini, modelos open-source (Mistral, LLaMA)	Madurez del ecosistema, API estable, coste razonable para prototipo.
Contenerización	Docker Compose v2	Kubernetes, Podman	Permite un despliegue sencillo y reproducible, suficiente para el alcance del TFM.

En cuanto a los costes asociados al uso de la API de OpenAI, el modelo gpt-5.4-mini ofrece un coste significativamente inferior al de modelos de mayor capacidad, lo que lo convierte en una opción adecuada para prototipos y fases iniciales de validación. Tomando como referencia las tarifas por tokens, el coste estimado de un análisis de contrato de tamaño moderado podría situarse aproximadamente entre 0,01 y 0,05 EUR, dependiendo de la extensión del documento y de la longitud de la respuesta generada. Del mismo modo, el coste por turno de conversación en un chatbot podría rondar entre 0,005 y 0,02 EUR. En consecuencia, estos costes pueden considerarse asumibles tanto para un proyecto de TFM como para una fase inicial de validación con usuarios reales.

3.4. Diseño de datos y pipelines

El diseño de datos de ContratIA se divide en dos partes: el esquema relacional en PostgreSQL y el pipeline del corpus RAG.

Esquema relacional

La base de datos relacional contiene las siguientes entidades principales:

- users: almacena usuarios con su email, hash de contraseña, nombre, apellidos, empresa, rol, estado de activación y marca temporal del último inicio de sesión.
- legal_acceptances: registro de aceptación de términos y política de privacidad, con versión de cada documento y timestamp de aceptación.
- contracts: metadatos de los contratos subidos como nombre original, ruta del fichero, tipo MIME, tamaño, número de páginas, estado, nivel de riesgo global, idioma detectado y tipo de contrato.
- contract_analyses: resultado agregado del análisis vinculado a un contrato registrando nivel de riesgo global, resumen narrativo, tokens consumidos, versión del prompt y estado del proceso.
- contract_clauses: cada cláusula extraída del contrato registra número, título, texto original, explicación en lenguaje llano, nivel de riesgo, consejo legal e índice de orden.
- contract_risks: riesgos formales identificados en cada cláusula.
- contract_aspects: aspectos generales detectados en cada cláusula, clasificados como positivos o negativos, pero sin asignarles un nivel de severidad.
- contract_missing_elements: elementos que deberían figurar en el contrato pero están ausentes..o
- contract_analysis_evidence: fragmentos del corpus legal RAG utilizados como evidencia durante el análisis.
- clause_relations: relaciones detectadas entre cláusulas.
- agent_conversations + agent_messages: conversaciones y mensajes del chatbot, asociados opcionalmente a un contrato.

- generated_contracts + generated_contract_versions: borradores de contratos generados con la aplicación, con historial versionado de cambios.
- user_monthly_usage: registro mensual por usuario de contratos analizados, contratos generados y modificaciones por IA, para control de cuotas.

Pipeline del corpus RAG

El corpus jurídico RAG se construye a través de un pipeline reproducible de cuatro etapas:

1. Obtención y normalización: los textos legales se descargan del BOE en formato HTML y se convierten a Markdown estructurado. Cada texto legal tiene un identificador oficial (BOE ID), una fecha de consolidación y un hash SHA-256 para verificar su integridad.
2. Segmentación (chunking): los textos normalizados se dividen en fragmentos de entre 250 y 900 tokens, respetando los límites de artículo y de párrafo para preservar la coherencia semántica. Cada fragmento almacena metadatos: ley origen, número de artículo, cita formal (ej. CC art. 1254) y hash del texto.
3. Embedding: cada fragmento se convierte en un vector de 1.536 dimensiones usando el modelo text-embedding-3-small de OpenAI. Los vectores se almacenan junto a los metadatos del fragmento.
4. Carga en Qdrant: los vectores y sus metadatos se cargan en una colección de Qdrant con índice de similitud coseno. El servicio qdrant-init verifica la integridad del corpus (hashes SHA-256) antes de cargar, rechazando cualquier corpus corrupto.

El corpus v2 contiene 13 documentos legales y 2.786 fragmentos. Los documentos incluidos son:

- Código Civil (BOE-A-1889-4763, consolidado a enero 2025)
- Ley de Arrendamientos Urbanos – LAU (BOE-A-1994-26003, consolidado a mayo 2023)
- Texto Refundido de la Ley General para la Defensa de los Consumidores y Usuarios – TRLGDCU

- Código de Comercio (BOE-A-1885-6627)
- Reglamento General de Protección de Datos – RGPD (UE 2016/679)
- Ley Orgánica de Protección de Datos y Garantía de Derechos Digitales – LOPDGDD (BOE-A-2018-16673)
- Ley de Servicios de la Sociedad de la Información – LSSI (BOE-A-2002-13758)
- Texto Refundido de la Ley de Propiedad Intelectual – TRLPI
- Ley de Condiciones Generales de la Contratación – LCGC
- Ley de Medidas de Lucha contra la Morosidad (BOE-A-2010-10708)
- Estatuto del Trabajo Autónomo (BOE-A-2007-13409)
- Ley 6/2020 de servicios electrónicos de confianza (eIDAS)
- Directiva 2011/83/UE sobre derechos de los consumidores

4. Implementación

4.1. Estructura del código y componentes

El código fuente completo del proyecto es de acceso público y está alojado en el repositorio de GitHub https://github.com/adrianvizga/EBIS_TFM_Contratia. El repositorio se organiza en tres directorios principales:

backend/

El backend está escrito en Python y se despliega en Docker sobre Python 3.11. Sigue una estructura organizada por responsabilidades:

- api/endpoints/: routers de FastAPI agrupados por dominio:

  auth.py: registro, login, Google ID token, refresh, perfil y gestión de contraseña.

  contracts.py: subida, listado, consulta del análisis, borrado, reintento y stream de estado.

  Generated_contracts.py: generación, edición, versionado, visualización y envío a análisis.

  agent.py: chatbot conversacional.

  notifications.py: Gestión de las notificaciones al usuario.

  usage.py: Control de los límites de uso de la aplicación.

  settings.py: Control de la configuración del usuario.

- services/: capa de lógica de negocio. Podemos encontrar:

  legal_analysis.py: análisis jurídico cláusula a cláusula y agregación.

  chat_service.py: chatbot con historial/memoria y contexto contractual.

  rag_service.py: recuperación RAG desde Qdrant.

  contract_generation.py: prompts, reglas y validación de borradores.

  openai_client.py: llamadas principales a OpenAI para extracción/generación/modificación.

clause_relations_service.py: detección de relaciones y referencias entre cláusulas.

- workers/tasks.py: tareas Celery. Está dividido en:

analyze_contract para la orquestación del análisis completo: extracción de texto, extracción inicial de cláusulas mediante OpenAI, análisis jurídico individual con RAG, agregación, detección de relaciones, persistencia en base de datos y notificaciones.

generate_contract para generar contratos en segundo plano.

- models/: modelos Pydantic. Encontraremos en este paquete:

contract_analysis.py: define modelos estrictos para validar respuestas estructuradas de OpenAI con extra='forbid'.

models.py contiene modelos de API para autenticación, usuario y settings.

- core/: configuración, base de datos, seguridad, inicialización de BD y logging. Todo gestionado por:

config.py: valida y normaliza variables relevantes

database.py: usa pool de conexiones PostgreSQL;

security.py: gestiona JWT y bcrypt;

db_initializer.py: aplica esquema base y migraciones versionadas;

logging_config.py: configura logging.

- rag_tools/: pipeline del corpus RAG dividido en descarga/refresh de fuentes oficiales, normalización, chunking, embeddings, carga en Qdrant, evaluación y validación.

frontend/

El frontend está escrito en TypeScript con React 19 y Vite 7. La estructura combina organización por tipo de módulo y por feature:

- pages/: páginas de autenticación y documentos legales: SignIn, SignUp, ResetPassword y LegalDocument (se comparte por /terms y /privacy).
- components/sections/: secciones principales de la aplicación: Dashboard, AnalyzeContract, ContractAnalysis, GenerateContract, EditGeneratedContract, ViewGeneratedContract, Agent y Settings.
- hooks/: custom hooks reutilizables como useAuthFetch, useSettings, useUsage y useContractTypes.
- components/: componentes compartidos como ClauseRelationsGraph, ConfirmDeleteModal, DownloadDropdown, ErrorModal, Header, Sidebar, LegalModal, ProtectedRoute, UsageBadge y UpgradePlanScreen.
- config/api.ts: centraliza el origen/base de la API mediante API_ORIGIN y API_BASE_URL. Otros ficheros de config gestionan Google, textos legales y traducciones.

backend/legal_corpus/

Directorio que contiene el corpus jurídico organizado por idioma. Actualmente el corpus español está en legal_corpus/es/, con versiones en subdirectorios de formato vN_AAAA-MM-DD. Cada versión contiene:

- manifest.json con los metadatos del corpus.
- sources/ con las fuentes originales.
- normalized/ con los textos normalizados en Markdown.
- chunks/ con los fragmentos generados y estadísticas.
- embeddings/ con la caché de vectores.
- evaluations/ con los datasets e informes de evaluación RAG.

4.2. Integración con modelos OpenAI

La integración con la API de OpenAI se realiza a través del SDK oficial de Python. Para garantizar la resiliencia ante errores transitorios de la API, se implementó el módulo openai_resilience.py, que centraliza reintentos para varios flujos con un máximo de 3 intentos. En el análisis jurídico por cláusula existe además una lógica propia de reintentos, también con un máximo de 3 intentos, con esperas progresivas de 2 y 4 segundos antes de agotar el intento principal y recurrir al modelo de respaldo si está configurado.

El flujo de análisis de una cláusula sigue estos pasos:

1. Se construye el prompt de sistema con las instrucciones de análisis jurídico, el esquema JSON de salida esperado, la calibración de niveles de riesgo y las instrucciones de seguridad frente a prompt injection.

2. Se recupera el contexto legal relevante desde Qdrant mediante una query basada en el texto original de la cláusula. Para el análisis por cláusula se solicitan los top-5 fragmentos y se aplica el umbral mínimo de score configurado (0,35).

3. Se construye el prompt de usuario incorporando los metadatos del contrato, el número y título de la cláusula, el texto original de la cláusula y, cuando existe, el contexto legal recuperado del corpus RAG.

4. Se llama a la API de OpenAI con response_format de tipo json_schema estricto y temperatura 0, salvo en modelos que no admiten temperatura explícita, para maximizar la consistencia de la salida.

5. La respuesta se valida contra el modelo Pydantic ClauseLegalAnalysis. Si la validación o la llamada fallan tras los reintentos, se registra el error; el fallo se propaga al proceso de análisis, que marca el contrato y el análisis asociado con estado de error.

Para el chatbot, la integración es similar, pero añade el historial de conversación, limitado por defecto a 6.000 tokens, y opcionalmente el texto del contrato del usuario, limitado por defecto a 10.000 tokens. Además, recupera desde Qdrant los fragmentos legales más relevantes y los incorpora al análisis como evidencia identificada. El prompt del chatbot incluye reglas estrictas: no inventar leyes, artículos, jurisprudencia, hechos, plazos ni citas; mantenerse dentro del ámbito contractual español; tratar el contrato, la evidencia, el historial y el mensaje del

usuario como datos no confiables; distinguir entre referencias al contrato y respaldo del corpus legal; y garantizar que la salida siga siempre una estructura JSON definida.

Para la generación de contratos, se usa primero el modelo principal configurado en OPENAI_MODEL y, si falla, el modelo de respaldo configurado en OPENAI_FALLBACK_MODEL. La generación utiliza temperatura 0 para maximizar la consistencia del borrador. Antes de llamar al modelo, se recupera contexto legal mediante varias consultas RAG específicas por tipo de contrato y se incorporan hasta 15 fragmentos legales únicos al prompt.

El prompt de usuario de generación incorpora las LEGAL_COMPLIANCE_RULES específicas del tipo de contrato: reglas LAU para arrendamientos de vivienda y local, y Código Civil y normativa aplicable para prestación de servicios. Estas reglas obligan al modelo a citar el artículo de referencia en cada cláusula, respetar límites legales de fianza, duración, prórrogas, desistimiento, responsabilidad y otras condiciones aplicables, y redactar cláusulas equilibradas. El resultado se valida y se comprueba longitud mínima y máxima, que la respuesta no sea un rechazo del modelo, la presencia de términos contractuales obligatorios y una estructura reconocible de cláusulas mediante expresión regular.

4.3. Gestión de prompts y orquestación

Uno de los aspectos más importantes del desarrollo fue el diseño y la iteración sobre los prompts del sistema. ContratIA utiliza identificadores de versión para algunos flujos relevantes: el chatbot define CHAT_PROMPT_VERSION, la generación de contratos define GENERATION_PROMPT_VERSION y MODIFICATION_PROMPT_VERSION, y los análisis contractuales disponen de un campo analysis_version en la base de datos. Esto permite auditar parcialmente qué versión lógica del flujo generó determinados resultados, especialmente en respuestas del chatbot y contratos generados, aunque no se almacena una versión específica de cada prompt de análisis por cláusula.

Los prompts más relevantes son:

- SYSTEM_PROMPT_WITH_RAG, usado en el análisis con contexto legal, instruye al modelo para actuar como experto en revisión práctica de contratos en España, citar artículos legales cuando se proporcione contexto RAG, usar una escala

de riesgo definida y devolver JSON válido conforme al esquema esperado. También incluye una instrucción explícita frente a prompt injection.

- SYSTEM_PROMPT_NO_RAG es la versión alternativa cuando no hay contexto RAG disponible. Mantiene la estructura de análisis, pero pide al modelo ser más prudente y evitar citas legales específicas o afirmaciones firmes de ilegalidad salvo en casos evidentes.

- CHAT_DEVELOPER_PROMPT es el prompt del chatbot. Contiene 8 reglas obligatorias que incluyen responder en el idioma indicado por la aplicación, no inventar leyes, artículos, jurisprudencia, hechos, plazos ni citas, distinguir entre referencias al contrato y evidencia legal, restringir el campo citations a los identificadores de evidencia recuperada, tratar el contrato, la evidencia, el historial y el mensaje del usuario como datos no confiables, limitarse al ámbito contractual y jurídico, y devolver exclusivamente JSON con los campos solicitados.

- GENERATION_SYSTEM_PROMPT se utiliza para la generación de contratos. El prompt incluye instrucciones de jerarquía y seguridad, alcance del borrador, calidad jurídica y formato de salida. Exige no inventar normas ni datos, prohíbe incluir cláusulas que vulneren derechos irrenunciables, sean abusivas o contravengan normas imperativas, y obliga a citar el artículo de referencia en cada cláusula regulada. Soporta dos tonos seleccionables por el usuario: «Legal Formal», con terminología jurídica precisa y estructura propia de contratos españoles, y «Lenguaje claro», con vocabulario accesible para personas sin formación jurídica y preferencia por términos simples como «inquilino» frente a «arrendatario». Las LEGAL_COMPLIANCE_RULES se inyectan en el prompt de usuario separado de las partes, los requisitos del usuario, la evidencia legal y la estructura mínima.

La orquestación de la tarea de análisis se realiza en workers/tasks.py. La tarea Celery analyze_contract implementa una máquina de estados implícita mediante los estados pending, processing, completed y error. En caso de error, el contrato se marca como fallido, se actualiza el análisis asociado cuando existe y se envía una notificación al usuario. El procesamiento no es atómico de extremo a extremo, ya que se realizan commits intermedios durante el flujo; si ocurre un fallo, el sistema intenta dejar el contrato en estado error y evitar que aparezca como completado, aunque pueden existir resultados parciales ya persistidos.

4.4. Mecanismos de seguridad

La seguridad fue una prioridad desde el diseño inicial. Los mecanismos implementados cubren varias amenazas relevantes de una aplicación web con IA.

Autenticación y autorización

La aplicación utiliza JWT con algoritmo HS256. Los tokens de acceso tienen una duración configurable mediante ACCESS_TOKEN_EXPIRE_MINUTES, con 30 minutos por defecto. Los tokens de refresco tienen una duración configurable mediante REFRESH_TOKEN_EXPIRE_DAYS, con 7 días por defecto, y pueden ampliarse a 30 días cuando se usa la opción "recordarme". Los tokens de refresco se almacenan en la base de datos en la tabla sessions, se validan en cada renovación y pueden revocarse eliminando la sesión asociada.

La aplicación integra Google OAuth para registro e inicio de sesión sin contraseña, validando el ID token de Google contra el GOOGLE_CLIENT_ID configurado.

Las contraseñas tienen requisitos mínimos de seguridad: al menos 8 caracteres, una mayúscula, una minúscula, un dígito y un máximo de 128 caracteres. Se almacenan mediante hashes bcrypt.

El control de acceso se aplica a nivel de recurso: los endpoints filtran por el identificador del usuario para que cada usuario solo pueda acceder a sus propios contratos, análisis, contratos generados y conversaciones.

Validación de entrada

Parte de los modelos Pydantic usan extra='forbid', especialmente los modelos estrictos de análisis y los modelos de generación/edición de contratos. Además, se aplican restricciones de longitud, tipos y valores permitidos en múltiples modelos de entrada.

Los ficheros subidos se validan por extensión permitida (.pdf, .docx, .txt), tamaño máximo, contenido básico del archivo y número máximo de páginas. En PDF y DOCX se comprueba la cabecera del contenido, y en TXT se rechaza contenido binario o no decodificable. El MIME declarado por el cliente se almacena, pero la validación principal no depende solo de él.

Las entradas textuales destinadas a flujos con IA se limitan mediante campos Pydantic, guardrails y límites configurados. En el chatbot, el mensaje del usuario tiene longitud máxima configurable, el historial se limita por tokens y el texto del contrato vinculado se trunca antes de enviarse al modelo.

Protección frente a prompt injection

Los prompts de sistema incluyen instrucciones explícitas que declaran el contrato del usuario, los fragmentos RAG, el historial y el mensaje del usuario como datos no confiables que no pueden modificar las reglas del sistema.

Además, existe un módulo de guardrails que detecta patrones habituales de prompt injection, como intentos de ignorar instrucciones previas, cambiar el rol del sistema, revelar el prompt o forzar una estructura de respuesta.

Los resultados del LLM se validan en los flujos críticos: el análisis contractual se valida contra modelos Pydantic estrictos y la generación de contratos se comprueba con validadores específicos de longitud, estructura y contenido mínimo.

Cabeceras de seguridad HTTP

El backend añade cabeceras de seguridad como X-Content-Type-Options: nosniff, X-Frame-Options: DENY, X-XSS-Protection, Referrer-Policy y Permissions-Policy.

En el despliegue frontend mediante Nginx se configura además Content-Security-Policy (CSP), restringiendo scripts, estilos, fuentes, imágenes, conexiones y frames a orígenes conocidos como la propia aplicación y Google para el inicio de sesión.

CORS está restringido a los orígenes configurados en CORS_ORIGINS. La configuración rechaza explícitamente wildcard (*) y orígenes null cuando las credenciales están habilitadas.

Rate limiting y cuotas

La aplicación usa SlowAPI como middleware de rate limiting en endpoints sensibles, incluyendo registro, login, refresh token, Google login, recuperación/restablecimiento de contraseña, subida/reintento de análisis y chat.

También existe un sistema de cuotas por plan y periodo mensual. Controla el número de contratos generados, contratos analizados y modificaciones con IA por contrato. El acceso al asistente jurídico depende del plan del usuario, y el uso del chat está además protegido por rate limiting.

4.5. Despliegue e infraestructura

ContratIA se despliega mediante Docker Compose v2. El fichero compose.yml define ocho servicios con sus dependencias, variables de entorno, healthchecks en los servicios principales y volúmenes persistentes.

Los servicios definidos se componen de postgres, redis, qdrant, backend, worker y frontend como servicios principales, y db-init, qdrant-init como servicios auxiliares. Las imágenes base principales son:

- Backend: Python (3.11.11-slim), ejecutado con usuario no root contratia:10001 para reducir la superficie de ataque.
- Worker Celery: reutiliza la misma imagen que el backend, pero con un comando de arranque distinto para ejecutar Celery.
- Frontend: build multi-stage con Node (22.14.0-alpine) para instalar dependencias, ejecutar tests y compilar, y Nginx (1.27.4-alpine) para servir la aplicación React.
- PostgreSQL, Redis y Qdrant: imágenes oficiales con versiones fijadas para PostgreSQL (15.12-alpine), Redis (7.4.2-alpine) y Qdrant (1.13.4).

El orden de arranque está gestionado mediante depends_on con condiciones y healthchecks. PostgreSQL debe estar listo antes de ejecutar db-init, y Qdrant debe estar listo antes de ejecutar qdrant-init. Backend y worker esperan a que db-init y qdrant-init finalicen correctamente, y también dependen de Redis en estado healthy. Los servicios db-init y qdrant-init se ejecutan como tareas de inicialización configuradas para no reiniciarse.

La persistencia de datos se gestiona mediante volúmenes Docker para PostgreSQL, Redis, Qdrant y los ficheros subidos por los usuarios. Los datos del corpus RAG forman parte del backend y se copian dentro de la imagen, por lo que no requieren un volumen externo específico.

Para entornos de producción, la configuración de la aplicación se externaliza mediante variables de entorno, definidas en un fichero .env o directamente en el entorno de despliegue. Entre estas variables se incluyen las claves de acceso a OpenAI, la SECRET_KEY utilizada para la generación de tokens JWT, las credenciales de PostgreSQL, la contraseña de Redis, la API key de Qdrant, la configuración de CORS_ORIGINS restringida al dominio de producción, el GOOGLE_CLIENT_ID necesario para la autenticación OAuth y las credenciales SMTP empleadas para el envío de correos, como los mensajes de restablecimiento de contraseña.

La monitorización en producción puede ampliarse con herramientas externas como Flower para Celery, Prometheus y Grafana para métricas de contenedores, y una solución de gestión de logs como Loki o Elasticsearch. El backend incluye una configuración de logging configurable por entorno, con salida a consola o fichero y soporte opcional para formato JSON, aunque estas herramientas de monitorización no están incluidas actualmente como servicios del compose.yml.

5. Evaluación y experimentos realizados

5.1. Métricas de evaluación

La evaluación del sistema se organizó principalmente en torno a la calidad del sistema RAG: recuperación de fragmentos legales, validez de las citas, abstención ante consultas no respondibles y coherencia del contexto recuperado.

Métricas del sistema RAG

El pipeline de evaluación RAG (rag_tools/evaluate_rag.py) calcula las siguientes métricas a partir de un dataset de casos de prueba definido en rag_eval_v2.json:

Métrica	Descripción	Umbral mínimo
Hit Rate	Proporción de casos en los que al menos un fragmento esperado está entre los recuperados.	>= 0,95
Recall	Proporción media de fragmentos esperados que se recuperan.	>= 0,90
Relevance Precision	Proporción media de fragmentos recuperados que son relevantes.	>= 0,80
Citation Accuracy	Proporción de citas recuperadas cuyo texto en Qdrant coincide exactamente con el hash esperado.	= 1,00
Faithfulness	Proporción de fragmentos del contexto que aparecen literalmente en la respuesta generada.	= 1,00
Abstention Rate	Proporción de casos «no respondibles» en los que el modelo abstiene correctamente.	>= 0,95
Context Accuracy	Proporción de casos sin recuperación de leyes prohibidas (exclusión por tipo de contrato).	= 1,00

Métricas del análisis de contratos

Para el análisis de contratos, el sistema registra y controla algunos indicadores operativos y de consistencia, aunque no existe un pipeline de evaluación separado equivalente al de RAG. Para los contratos se evalúa:

- Coherencia del nivel de riesgo: el flujo de análisis ajusta el riesgo global final para que sea coherente con la distribución de riesgos detectados en las cláusulas individuales. De este modo, el resultado agregado del modelo se limita dentro de un rango razonable derivado de los riesgos por cláusula.

- Validación de respuestas JSON: las respuestas del LLM en el análisis estructurado se validan contra modelos Pydantic estrictos. Si una respuesta no

supera la validación, el flujo reintenta la llamada y, si el error persiste, marca el análisis como fallido.

- Duración del análisis: la tabla contract_analyses registra started_at y finished_at, lo que permite estimar la duración del procesamiento del análisis.

- Consumo de tokens: el sistema acumula los tokens utilizados durante las fases de análisis y los persiste en el campo tokens_used. Esto permite estimar posteriormente el coste por análisis si se combina con la tarifa del modelo utilizado, aunque el coste económico no se calcula automáticamente en la aplicación.

5.2. Resultados cuantitativos y cualitativos

Los resultados del corpus RAG v2 se obtuvieron ejecutando el evaluador sobre el dataset de evaluación incluido en el corpus (evaluations/rag_eval_v2.json). Los resultados superan todos los umbrales definidos:

Métrica	Resultado obtenido	Umbral mínimo	¿Pasa?
Hit Rate	1,00 (100 %)	>= 0,95	Sí
Recall	1,00 (100 %)	>= 0,90	Sí
Relevance Precision	1,00 (100 %)	>= 0,80	Sí
Citation Accuracy	1,00 (100 %)	= 1,00	Sí
Faithfulness	1,00 (100 %)	= 1,00	Sí
Abstention Rate	1,00 (100 %)	>= 0,95	Sí
Context Accuracy	1,00 (100 %)	= 1,00	Sí

Estos resultados indican que el pipeline de corpus y el sistema de recuperación funcionan correctamente sobre el dataset de evaluación RAG v2. Se recuperan los fragmentos esperados, las citas son exactas, la respuesta construida por el evaluador es fiel al contexto recuperado y el sistema se abstiene correctamente cuando la pregunta no tiene respaldo en el corpus.

Además de la evaluación automática del RAG, se realizaron pruebas funcionales con contratos de ejemplo de los tipos soportados: arrendamiento de vivienda, arrendamiento de local y prestación de servicios. Estas pruebas permitieron comprobar cualitativamente que el sistema identifica riesgos por cláusula, genera explicaciones en lenguaje claro, incorpora referencias legales cuando el contexto RAG aporta respaldo suficiente y mantiene la coherencia entre el riesgo global y los riesgos individuales.

En el chatbot jurídico se verificó el comportamiento esperado ante consultas contractuales vinculadas al corpus con recuperación de evidencia legal, citas limitadas a los fragmentos recuperados y abstención cuando no existe respaldo suficiente.

En la generación de contratos, las pruebas funcionales confirmaron que los borradores incluyen una estructura contractual completa, con cláusulas como objeto, duración, renta o precio, fianza cuando procede, gastos, conservación u obligaciones de las partes, resolución, notificaciones, ley aplicable y firmas. El resultado se valida posteriormente mediante comprobaciones de longitud, estructura y presencia de términos contractuales mínimos.

En cuanto al rendimiento, el sistema registra información operativa como timestamps de inicio y finalización del análisis y tokens consumidos, lo que permite estimar duración y coste.

5.3. Análisis de errores y limitaciones

Durante el desarrollo y las pruebas se identificaron varios casos de error y limitaciones del sistema:

- Documentos con texto no seleccionable (PDFs escaneados): el sistema no puede extraer texto de imágenes y devuelve un error claro al usuario indicando que el documento no contiene texto seleccionable. La incorporación de OCR queda como trabajo futuro.

- Contratos mixtos o complejos: contratos que mezclan varios tipos (ej. arrendamiento con opción de compra) pueden ser clasificados incorrectamente o generar un análisis parcial. El sistema detecta cuando el tipo de contrato está fuera del corpus con una puntuación RAG por debajo del umbral y lo indica al usuario.

- Alucinaciones residuales: aunque el RAG y las instrucciones del prompt reducen significativamente las alucinaciones, no las eliminan por completo. En algunas respuestas del chatbot se observaron citas de artículos correctos pero con una descripción ligeramente imprecisa. La validación de citas en el pipeline de evaluación no cubre la precisión semántica de la descripción, solo la exactitud del identificador.

- Ambigüedad en la escala de riesgos: la escala de riesgo está definida en el prompt, pero la calibración entre «medium» y «high» puede variar entre análisis. Se observaron casos en los que cláusulas similares recibían niveles distintos dependiendo del contexto del contrato.

- Cobertura limitada del corpus: el corpus cubre los tipos de contrato más habituales, pero algunos contratos válidos en España (ej. contratos de agencia, franquicia, licencia de software) no tienen soporte en el corpus y el sistema abstiene o proporciona respuestas genéricas.

6. Pruebas y validación de producto

6.1. Tests unitarios y funcionales

El proyecto incluye pruebas tanto para el backend como para el frontend. En el backend, las pruebas se implementan con pytest y pytest-asyncio. También está disponible pytest-cov para generar informes de cobertura cuando se ejecutan los tests con las opciones correspondientes.

Las categorías de tests incluyen:

- Tests de servicios: verifican el comportamiento de servicios como análisis, chatbot y generación utilizando mocks de dependencias externas, incluida la API de OpenAI, para comprobar la lógica de validación, parsing y manejo de errores sin depender de la disponibilidad real de la API.

- Tests de integración: verifican flujos de la aplicación mediante TestClient y tareas/módulos del backend, aunque en general utilizan mocks de base de datos y de servicios externos. No existe una suite end-to-end completa ejecutada contra una base de datos PostgreSQL real desde la carga de un fichero hasta la obtención final del análisis.

- Tests del corpus RAG: el proyecto incluye herramientas de evaluación del pipeline RAG, como evaluate_rag.py y validate_rag.py, que calculan métricas y comprueban umbrales mínimos de calidad. Estas herramientas funcionan como validación del corpus y del sistema de recuperación.

- Tests de seguridad: existen pruebas relacionadas con autenticación, generación y verificación de JWT, hash de contraseñas, cabeceras de seguridad, rate limiting y validaciones de configuración como CORS y proxy.

En el frontend, las pruebas se implementan con Vitest y Testing Library. Se cubren hooks personalizados y componentes relevantes de la interfaz. MSW (Mock Service Worker) se usa para interceptar llamadas a la API durante los tests, evitando la necesidad de un backend real.

6.2. UX y usabilidad

El diseño de la interfaz de usuario partió de la premisa de facilitar el uso a personas sin formación jurídica ni técnica avanzada. Esta orientación se refleja en el uso de lenguaje claro, explicaciones por cláusula, recomendaciones accionables y una presentación visual de riesgos y estados.

Las decisiones de UX más relevantes fueron:

- Carga de contratos: la pantalla de análisis permite subir contratos mediante selector de fichero o drag & drop, mostrando errores de carga cuando procede. Los contratos analizados se presentan en un historial con filtros, paginación, acciones y etiquetas visuales de estado: pendiente, procesando, completado y error.

- Presentación de resultados: el análisis se organiza en un resumen del contrato, nivel global de riesgo, listados de riesgos, elementos ausentes, aspectos positivos y, cuando existen, aspectos negativos. Las cláusulas se muestran en una lista desplegable con texto técnico, explicación en lenguaje claro y recomendaciones cuando existe riesgo.

- Grafo de relaciones: las relaciones entre cláusulas representan contradicciones, dependencias, duplicidades y referencias a cláusulas ausentes. El grafo permite inspeccionar visualmente la complejidad estructural del contrato.

- Tema claro/oscuro: la interfaz soporta tema claro y oscuro, configurable desde ajustes y persistido en la configuración del usuario.

- Multiidioma: la aplicación permite seleccionar español, inglés, francés, alemán y portugués desde la configuración del usuario. El chatbot utiliza esta preferencia para responder en el idioma indicado por la aplicación.

- Advertencias contextuales: el chatbot devuelve respuestas estructuradas con answer, citations, limitations y abstained. El prompt le exige no inventar respaldo legal, abstenerse cuando la evidencia no permita sostener una conclusión y añadir una advertencia de revisión profesional cuando la respuesta sea incompleta o el caso requiera valoración individual.

Durante el desarrollo se puede destacar que el campo de recomendación o consejo legal de cada cláusula está diseñado para traducir la implicación jurídica a una acción comprensible para el usuario.

6.3. Observabilidad en producción

La observabilidad es un aspecto importante en aplicaciones que integran IA, ya que los errores pueden ser sutiles y la latencia puede variar significativamente.

ContratIA implementa varios mecanismos de observabilidad:

- Logging centralizado: el backend utiliza el módulo logging_config.py para configurar el logging de forma común en la API, scripts y worker. La salida puede configurarse por entorno para consola, fichero o ambos, y el formato puede ser texto o JSON.

- Request ID HTTP: cada petición HTTP recibe un identificador único corto, derivado de un UUID, que se incluye en los logs de entrada/salida de la petición y se devuelve al cliente mediante la cabecera X-Request-ID.

- Registro transaccional de conversaciones: cada turno del chatbot se guarda junto con el request_id de la solicitud, la versión del prompt, el modelo usado, los tokens de entrada y salida, las evidencias/citas, las limitaciones y el indicador de abstención. El guardado del mensaje del usuario, la respuesta del asistente y la actualización de la conversación se realiza dentro de una transacción de base de datos.

- Notificaciones al usuario: el sistema crea notificaciones internas cuando un análisis o una generación se completa o falla. Estas notificaciones complementan el seguimiento del estado desde el frontend.

- Healthchecks: el backend expone /health y /ready, y Docker Compose utiliza /ready para comprobar disponibilidad real de PostgreSQL, Redis y Qdrant desde la API. PostgreSQL, Redis, Qdrant, Celery y frontend también tienen healthchecks definidos en compose.yml.

- Monitorización externa: Celery podría monitorizarse añadiendo herramientas externas como Flower.

7. Discusión

7.1. Lecciones aprendidas durante el desarrollo

El desarrollo de ContratIA a lo largo del máster ha sido un proceso de aprendizaje intenso en múltiples dimensiones. A continuación se recogen las lecciones más relevantes.

La ingeniería de prompts es un proceso iterativo y exige rigor

Los primeros prompts producían resultados inconsistentes: a veces el modelo devolvía texto libre en lugar de JSON, otras veces incluía campos no esperados, y en algunos casos el nivel de riesgo no era coherente con la explicación. La solución fue definir esquemas JSON muy estrictos, validarlos con Pydantic y versionar los prompts para poder comparar el comportamiento entre versiones. El paso más determinante fue añadir el esquema JSON directamente en el system prompt con comentarios de los valores válidos para cada campo.

El corpus RAG es tan bueno como su proceso de normalización

La calidad de la recuperación RAG dependió críticamente de cómo se procesaron los textos legales. Los primeros intentos de chunking basados simplemente en número de tokens producían fragmentos que cortaban artículos a mitad, lo que dañaba la coherencia semántica y bajaba los scores de recuperación. La solución fue desarrollar un chunker específico que respeta los límites de artículo y de párrafo, con un rango de 250-900 tokens por fragmento, con lo que mejoró significativamente el recall de la evaluación.

La asincronía es imprescindible pero añade complejidad

Procesar contratos de forma síncrona en la petición HTTP es inviable: el análisis de un contrato de 10 páginas puede tardar 2-3 minutos. La introducción de Celery y Redis fue necesaria, pero añadió complejidad al sistema (gestión de estados, reintentos, transacciones atómicas). Una lección importante fue que el estado de la tarea debe ser siempre consistente con el de la base de datos: si la escritura de resultados falla, la tarea debe hacer rollback y notificar el error al usuario, no quedar en un estado inconsistente.

La seguridad de un sistema con LLMs requiere atención especial al prompt injection

Durante las pruebas se verificó que sin las instrucciones de prompt injection, era posible insertar instrucciones en el texto del contrato que modificaban el comportamiento del modelo. La solución fue declarar explícitamente en el system prompt que el contrato y los fragmentos RAG son datos no confiables, junto con la validación Pydantic estricta de la salida.

El tipo de contrato como filtro es esencial para la calidad del RAG

En las primeras versiones del corpus, la recuperación no filtraba por tipo de contrato. Esto producía que un contrato de prestación de servicios recuperase fragmentos de la LAU irrelevantes para el caso. Se añadió el campo contract_type como filtro en las queries de Qdrant, junto con un campo forbidden_laws en los casos de evaluación para detectar recuperaciones incorrectas.

Docker Compose simplifica el onboarding pero esconde la complejidad de producción

Tener todo en Docker Compose hace que el proyecto sea fácil de arrancar pero también oculta la complejidad que aparece en producción real: gestión de secretos, backups de la base de datos, actualizaciones de contenedores sin downtime, escalado horizontal del worker Celery. Para un TFM esto es aceptable, pero un despliegue real requeriría migrar a una orquestación más robusta como Kubernetes o servicios gestionados en la nube.

7.2. Riesgos, ética y mitigaciones

El desarrollo de una aplicación de análisis legal con IA plantea riesgos específicos que es importante analizar y mitigar.

Riesgo	Descripción	Mitigación implementada
Alucinaciones legales	El LLM puede generar una referencia legal incorrecta, inexistente o no suficientemente respaldada.	Sistema RAG que ancla las respuestas en un corpus jurídico versionado. Evaluación del pipeline RAG con métricas de precisión de citas, fidelidad y abstención. En el chatbot, las citas se filtran contra la evidencia recuperada y el sistema se abstiene cuando el corpus no aporta respaldo suficiente.
Sustitución del asesoramiento legal	El usuario puede interpretar las respuestas de ContratIA como consejo legal vinculante.	Disclaimers en la interfaz, en los términos y condiciones y en el chatbot. La aplicación indica que los resultados son informativos, pueden

		contener errores u omisiones y no sustituyen la revisión de un profesional jurídico.
Sesgo en el análisis	El modelo puede valorar de forma desigual cláusulas que favorecen a una u otra parte contractual.	Los prompts incluyen reglas de calibración, prudencia y antisesgo, y obligan a evaluar el impacto real de cada cláusula. Las respuestas se apoyan en el corpus legal recuperado y en esquemas estructurados, reduciendo la generación libre de opiniones no fundamentadas.
Prompt injection	Un usuario malicioso puede insertar instrucciones en el contrato o en el mensaje para manipular el análisis.	Prompts con instrucciones explícitas anti-injection que tratan contratos, historial y evidencia como datos no confiables. Guardrails de entrada para patrones de inyección. Validación estructurada mediante JSON Schema y modelos Pydantic estrictos. Límites de longitud y recorte de contexto en el chatbot.
Filtración de datos de otros usuarios	Un usuario podría intentar acceder a contratos, conversaciones o resultados de otro usuario.	Autenticación mediante tokens JWT firmados con SECRET_KEY. Control de acceso a nivel de recurso mediante verificación de user_id en las consultas de contratos, contratos generados, conversaciones, notificaciones y ajustes.
Desactualización del corpus	Una norma del corpus puede ser modificada y el sistema seguir citando una versión anterior.	Corpus legal versionado con fecha de generación, recuperación y consolidación de fuentes. Manifiesto con hashes SHA-256 para detectar cambios no previstos. Herramientas de validación, refresco y regeneración del corpus, permitiendo actualizarlo y recargarlo sin modificar el código de la aplicación.
Dependencia de un proveedor externo	Si OpenAI sufre una interrupción, las funcionalidades de IA pueden verse afectadas.	Reintentos con backoff exponencial ante errores transitorios. Modelo alternativo configurable para ciertos flujos.

8. Conclusiones y trabajo futuro

ContratIA es el resultado de aplicar de forma integrada los conocimientos adquiridos durante el Máster en Ingeniería y Desarrollo de Soluciones con Inteligencia Artificial Generativa como el diseño de sistemas RAG, ingeniería de prompts, desarrollo de APIs, despliegue de aplicaciones modulares en contenedores Docker y principios de seguridad en aplicaciones con LLMs.

Los objetivos planteados al inicio del proyecto se han cumplido en su totalidad. La aplicación ofrece las tres funcionalidades previstas (análisis, chatbot y generación de contratos), el corpus RAG supera los umbrales de calidad definidos en el dataset de evaluación, el sistema de seguridad cubre las principales amenazas de este tipo de aplicaciones, y el despliegue con Docker Compose es reproducible desde cero.

La principal conclusión técnica del proyecto es que el enfoque RAG sobre corpus legal oficial es la estrategia correcta para este caso de uso: permite actualizar el conocimiento legal sin reentrenar el modelo, garantiza trazabilidad de las respuestas, reduce las alucinaciones y puede evaluarse de forma cuantitativa. El coste de mantener el corpus actualizado es manejable (actualización manual de textos del BOE) y la calidad de la recuperación es alta.

La principal conclusión de producto es que la utilidad real de la aplicación está en la traducción de la terminología legal a explicaciones comprensibles para usuarios no expertos. No se trata de reemplazar al abogado, sino de reducir la asimetría de información que existe entre el redactor de un contrato y el firmante. En este sentido, ContratIA cumple su objetivo.

Trabajo futuro

Las principales líneas de trabajo futuro identificadas son:

- Incorporación de OCR: integrar un motor de reconocimiento óptico de caracteres para poder analizar contratos en formato imagen o PDF escaneado.
- Ampliación del corpus a más tipos de contratos: añadir legislación específica de contratos de agencia, distribución, franquicia, licencias de software y contratos laborales, para ampliar el alcance de la aplicación.

- Fine-tuning del modelo de análisis: una vez que se disponga de un dataset suficientemente grande de análisis validados por profesionales legales, explorar el fine-tuning de un modelo base para mejorar la consistencia del análisis.
- Comparación de versiones de contratos: funcionalidad para comparar dos versiones de un mismo contrato e identificar los cambios con sus implicaciones legales.
- Integración con servicios de firma electrónica: permitir que el usuario firme electrónicamente el contrato generado desde la propia plataforma (integración con DocuSign).
- Modelos open-source como alternativa: explorar el uso de modelos como Mistral o LLaMA desplegados on-premise para reducir la dependencia de la API de OpenAI y eliminar el coste por token.
- Panel de administración: interfaz para gestionar usuarios, cuotas y el corpus legal, actualmente solo accesible a través de la base de datos directamente.
- Internacionalización del corpus: ampliar el corpus a otras jurisdicciones para dar soporte a contratos en otros países.
- Despliegue en Kubernetes: migrar de Docker Compose a un cluster de Kubernetes para obtener alta disponibilidad, escalado automático del worker y actualizaciones sin downtime.

9. Bibliografía

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. y Polosukhin, I. (2017). Attention Is All You Need. En Advances in Neural Information Processing Systems (NeurIPS 2017), 30.

[2] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... y Amodei, D. (2020). Language Models are Few-Shot Learners. En NeurIPS 2020.

[3] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... y Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. En NeurIPS 2020.

[4] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., ... y Stiennon, N. (2022). Training language models to follow instructions with human feedback. En NeurIPS 2022.

[5] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... y Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. En ICLR 2022.

[6] OpenAI. (2025). GPT-5 System Card. Recuperado de https://openai.com/index/gpt-5-system-card/

[7] Qdrant Team. (2024). Qdrant: Vector Database for the Next Generation of AI Applications. Recuperado de https://qdrant.tech.

[8] FastAPI. (2024). FastAPI Documentation. Recuperado de https://fastapi.tiangolo.com.

[9] Celery Project. (2024). Celery - Distributed Task Queue. Recuperado de https://docs.celeryq.dev.

[10] OWASP Foundation. (2023). OWASP Top 10 for Large Language Model Applications. Recuperado de https://owasp.org/www-project-top-10-for-large-language-model-applications/.

[11] Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016 (RGPD). Diario Oficial de la Unión Europea, L 119, 1-88.

[12] Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y Garantía de los Derechos Digitales (LOPDGDD). BOE núm. 294, de 6 de diciembre de 2018.

[13] Ley 29/1994, de 24 de noviembre, de Arrendamientos Urbanos (LAU). BOE-A-1994-26003.

[14] Real Decreto de 24 de julio de 1889 por el que se publica el Código Civil. BOE-A-1889-4763.

[15] Real Decreto de 22 de agosto de 1885 por el que se publica el Código de Comercio. BOE-A-1885-6627.

[16] Chiang, W-L., Zheng, L., Sheng, Y., Angelopoulos, A. N., Li, T., Li, D., ... y Stoica, I. (2024). Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference. arXiv:2403.04132.

[17] Zheng, L., Chiang, W-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., ... y Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv:2306.05685.

[18] Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., ... y Liang, P. (2021). On the Opportunities and Risks of Foundation Models. arXiv:2108.07258.

[19] Thirunavukarasu, A. J., Ting, D. S. J., Elangovan, K., Gutierrez, L., Tan, T. F. y Ting, D. S. W. (2023). Large language models in medicine. Nature Medicine, 29, 1930-1940.

[20] Nay, J. J. (2023). Law Informs Code: A Legal Informatics Approach to Aligning Artificial Intelligence with Humans. Northwestern Journal of Technology and Intellectual Property.

Anexo A. Seguridad en profundidad

A.1. Flujo de autenticación

El sistema de autenticación de ContratIA combina autenticación local con email y contraseña, autenticación mediante Google Sign-In y tokens JWT para mantener la sesión del usuario.

El usuario puede iniciar sesión enviando sus credenciales al endpoint POST /api/auth/login, o autenticarse con Google mediante POST /api/auth/google-login. En el caso de Google, el backend verifica el ID token con la librería oficial de Google y comprueba que el emisor sea válido.

Si la autenticación es correcta, el backend genera un access_token JWT firmado con HS256 y un refresh_token JWT. Por defecto, el access_token tiene una validez de 30 minutos. El refresh_token tiene una validez de 7 días, ampliable a 30 días cuando el usuario activa la opción de recordar sesión.

El access_token se devuelve al cliente y se mantiene en memoria durante la sesión de la aplicación, evitando almacenarlo en localStorage. El refresh_token se almacena en la tabla sessions de PostgreSQL y se envía al navegador como cookie HttpOnly, con Secure configurable, SameSite=Lax y path limitado a /api/auth.

Cuando el access_token caduca, el cliente llama a POST /api/auth/refresh. El backend valida el refresh_token recibido por cookie, comprueba que exista una sesión activa en base de datos y genera un nuevo access_token. Además, rota el refresh_token: elimina el token anterior de la tabla sessions, crea uno nuevo y actualiza la cookie.

Al cerrar sesión mediante POST /api/auth/logout, el backend elimina la sesión asociada al refresh_token y borra la cookie. En operaciones sensibles como cambio o restablecimiento de contraseña, el sistema elimina todas las sesiones del usuario, invalidando el acceso desde otros dispositivos.

Este diseño permite mantener access tokens de corta duración, revocar refresh tokens desde base de datos y reducir el impacto de una posible exposición del token de acceso.

A.2. Validación de entradas en la cadena RAG

Un riesgo específico de los sistemas RAG es que el contenido del corpus, el texto del contrato o el propio mensaje del usuario intenten modificar el comportamiento del LLM mediante prompt injection indirecto. ContratIA aplica varias defensas para reducir este riesgo.

Separación semántica:

Los prompts separan las instrucciones del sistema del contenido no confiable. En el chatbot, la evidencia legal recuperada se introduce dentro de la sección EVIDENCIA_LEGAL_NO_CONFIABLE y el contrato vinculado dentro de CONTRATO_NO_CONFIABLE. En el análisis contractual, el prompt distingue entre metadatos, cláusula a analizar y contexto legal recuperado.

Declaración de no confianza:

Los prompts indican explícitamente que el contrato, la evidencia RAG, el historial y las entradas del usuario deben tratarse como datos, no como instrucciones. Esta medida no elimina por completo el riesgo de prompt injection, pero reduce la probabilidad de que el modelo obedezca instrucciones incrustadas en documentos o fragmentos recuperados.

Escape y delimitación:

En el chatbot, el contenido no confiable se escapa antes de insertarse en el prompt, sustituyendo caracteres como <, > y &. Esto evita que un contrato o fragmento legal malicioso pueda cerrar etiquetas internas como CONTRATO_NO_CONFIABLE o EVIDENCIA_LEGAL_NO_CONFIABLE.

Validación de salida:

Las respuestas estructuradas de análisis se validan mediante esquemas JSON estrictos y modelos Pydantic con extra='forbid'. Si el modelo devuelve campos inesperados o una estructura inválida, la respuesta se rechaza. En el chatbot, la respuesta se exige en formato JSON y se validan los campos principales; además, las citas se filtran para aceptar solo identificadores de evidencia recuperada realmente disponibles.

Truncado de contexto:

Cuando un contrato se usa como contexto del chatbot, su contenido se limita mediante chat_max_contract_tokens, configurado por defecto en 10.000 tokens. También se limita el historial conversacional y la longitud máxima de la consulta del usuario. Esto reduce la superficie de entrada que podría contener instrucciones maliciosas o irrelevantes.

Anexo B. Pipeline RAG: detalle técnico

B.1. Proceso de normalización de textos legales

Los textos legales del BOE se almacenan en el corpus en formato HTML. Para poder procesarlos de forma homogénea, el módulo normalize_official_sources.py convierte estos documentos a un formato Markdown estructurado uniforme, identificado en el manifiesto como structured_markdown_v2.

El proceso de normalización incluye:

Limpieza del HTML:

Se ignoran elementos que no forman parte del contenido jurídico principal, como scripts, estilos, formularios, navegación y pies de página. También se descartan bloques auxiliares no útiles para el corpus.

Extracción de contenido jurídico:

El parser identifica encabezados y texto legal a partir de etiquetas HTML y clases habituales en los documentos oficiales, como libro, título, capítulo, sección, artículo, párrafo y apartado.

Conversión a Markdown estructurado:

Los encabezados generales se convierten en títulos Markdown y los artículos se representan con encabezados estandarizados de tipo "## Artículo …". Esto permite que el chunker detecte unidades legales como artículos, disposiciones y anexos.

Normalización básica del texto:

El contenido extraído se decodifica en UTF-8, se limpian espacios repetidos y se unifica la separación entre bloques de texto. El resultado se guarda como fichero .md dentro del directorio normalized.

Verificación de integridad:

Durante la normalización se calcula el hash SHA-256 del HTML fuente y del Markdown normalizado. Estos valores se almacenan en el manifest.json como source_sha256 y normalized_sha256, junto con el estado de normalización y el formato utilizado.

El formato structured_markdown_v2 permite que el chunker procese de forma uniforme los documentos legales incluidos en el corpus, conservando su estructura

básica y generando fragmentos trazables por artículo, fuente, versión y hash de texto.

B.2. Estrategia de chunking

La estrategia de chunking es uno de los factores más importantes para la calidad de la recuperación RAG. Un chunk demasiado pequeño puede perder contexto jurídico, mientras que un chunk demasiado grande puede mezclar información de distintos artículos o materias.

ContratIA utiliza un chunker jurídico basado en la estructura del documento normalizado. El chunker identifica unidades legales como artículos, disposiciones y anexos a partir de los encabezados Markdown generados en la fase de normalización.

Nivel de unidad legal:

Cada artículo, disposición o anexo se toma como unidad base. Si su contenido no supera el límite configurado de 900 tokens, se almacena como un único chunk.

División de unidades largas:

Cuando una unidad legal supera los 900 tokens, se divide por bloques de texto o párrafos consecutivos. El chunker agrupa párrafos hasta aproximarse al límite máximo sin sobrepasarlo. En las partes posteriores se vuelve a incluir el encabezado de la unidad para conservar el contexto jurídico.

Cada chunk almacena metadatos como:

law_key: identificador de la norma, por ejemplo CC_ES, LAU_1994 o RGPD.

law_name: nombre de la norma.

article_number: número del artículo o identificador de la disposición.

article_title: título del artículo, si existe.

unit_type: tipo de unidad, como article o provision.

structure_path: ruta jerárquica dentro de la norma, por ejemplo libro, título, capítulo o sección.

citation: cita formal del fragmento, por ejemplo CC art. 1254 o LAU art. 36. Si un artículo se divide, se añade “parte 1”, “parte 2”, etc.

text_hash: hash SHA-256 del texto normalizado del chunk.

token_count: número de tokens calculado con cl100k_base.

doc_version, jurisdiction, language, source_url y retrieved_at: metadatos de trazabilidad del corpus.

Los chunks se almacenan en chunks.jsonl y posteriormente se cargan en Qdrant. El payload de cada vector conserva estos metadatos, lo que permite filtrar y auditar los resultados recuperados por ley, artículo, versión del corpus o fuente legal.

B.3. Generación de embeddings

Los embeddings del corpus se generan con el modelo text-embedding-3-small de OpenAI, configurado por defecto mediante la variable OPENAI_EMBEDDING_MODEL. En el corpus v2, los vectores generados tienen 1.536 dimensiones.

El proceso de embedding incluye:

Preparación del texto:

Cada chunk se envía al modelo utilizando su campo text, que contiene el fragmento legal normalizado, normalmente con el encabezado del artículo incluido. La cita formal y el resto de metadatos se conservan como payload del chunk, pero no se concatenan explícitamente al texto enviado al modelo.

Caché de embeddings:

Los embeddings se almacenan en embeddings_cache.jsonl asociados al text_hash de cada chunk y al modelo utilizado. Esto permite reutilizar vectores ya generados si el texto no ha cambiado, evitando llamadas innecesarias a la API.

Llamada a la API:

Los chunks sin embedding en caché se envían a la API de embeddings de OpenAI en batches de 128 elementos. El proceso registra los nuevos embeddings y los guarda incrementalmente en la caché.

Almacenamiento vectorial:

Una vez disponibles los embeddings, los pares formados por vector y metadatos del chunk se cargan en Qdrant mediante qdrant_loader.py. La colección se configura con distancia coseno, adecuada para búsquedas de similitud semántica.

Dimensión y coste:

El corpus v2 contiene 2.786 chunks y aproximadamente 503.076 tokens según el conteo almacenado en token_count. Con el precio de text-embedding-3-small de 0,02 USD por millón de tokens, el coste estimado de generar los embeddings del corpus completo es de aproximadamente 0,01 USD.

Anexo C. Evolución del proyecto durante el desarrollo

C.1. De la idea inicial al prototipo funcional

El proyecto ContratIA no nació con la forma final que tiene hoy. La previsión inicial era más limitada: desarrollar una herramienta centrada únicamente en el análisis de contratos. Posteriormente, durante la evolución del proyecto, se incorporó un chatbot jurídico como apoyo a la consulta de dudas legales y, en una fase final, se añadió el generador de contratos. La transición hasta el sistema actual se produjo a través de varias iteraciones relevantes.

Primera iteración (meses 1-2): análisis básico de contratos.

La primera versión del proyecto se centró en validar la funcionalidad principal prevista: permitir al usuario subir un contrato y obtener un análisis automático de su contenido. Los primeros prototipos utilizaban llamadas directas a la API de GPT para identificar cláusulas relevantes, posibles riesgos y aspectos mejorables. Aunque los resultados eran útiles como prueba de concepto, también mostraban limitaciones importantes: el modelo podía ofrecer interpretaciones demasiado generales, omitir detalles jurídicos relevantes o formular observaciones sin suficiente justificación normativa. Esta fase permitió confirmar la viabilidad de la idea, pero también mostró la necesidad de estructurar mejor el análisis y apoyarlo en fuentes legales verificables.

Segunda iteración (meses 2-3): mejora del análisis e introducción del RAG.

Para aumentar la fiabilidad jurídica del sistema, se construyó el primer corpus legal del proyecto, el corpus v1, en febrero de 2026. Este corpus incluía los textos más relevantes para el análisis de contratos de arrendamiento, especialmente el Código Civil, en sus apartados relativos a obligaciones y contratos, y la Ley de Arrendamientos Urbanos. La incorporación de RAG permitió que las respuestas del modelo estuvieran mejor fundamentadas y redujo la tendencia a generar afirmaciones no verificadas. Sin embargo, el alcance seguía siendo limitado, ya que las consultas relacionadas con otros tipos de contratos, como servicios o compraventa, todavía no ofrecían el mismo nivel de fiabilidad.

Tercera iteración (meses 3-4): incorporación del chatbot jurídico.

Una vez consolidada la primera versión del análisis contractual, se añadió un chatbot jurídico para que el usuario pudiera plantear dudas sobre contratos y recibir respuestas contextualizadas. El primer prototipo del chatbot enviaba directamente las preguntas del usuario a la API de GPT con un prompt de sistema genérico sobre derecho español. Aunque las respuestas eran fluidas y convincentes, el modelo cometía errores en detalles legales concretos, como plazos, artículos o excepciones. Esto confirmó la importancia de conectar también el chatbot con el corpus legal mediante RAG, de forma que sus respuestas estuvieran ancladas en fuentes normativas y no dependieran únicamente del conocimiento interno del modelo.

Cuarta iteración (meses 4-5): consolidación del análisis y generación de contratos.

Durante esta fase se reforzó el módulo de análisis de contratos. Fue necesario diseñar un flujo asíncrono con Celery, ya que los primeros intentos de procesar los contratos dentro de la propia petición HTTP provocaban problemas de timeout. También se definió un esquema de datos más preciso para almacenar los resultados cláusula a cláusula.

Con el análisis y el chatbot funcionando, se añadió el generador de contratos como tercer pilar de la aplicación. Este módulo resultó técnicamente más sencillo que el análisis, porque partía de formularios estructurados y no requería el mismo uso intensivo de RAG. No obstante, exigió diseñar una validación robusta de los borradores generados para evitar textos incompletos, incoherentes o respuestas que no tuvieran realmente formato contractual.

Quinta iteración y cierre (mes 6): corpus v2, seguridad, evaluación y calidad.

La última iteración se centró en elevar la calidad global del sistema. El corpus v1 presentaba varias limitaciones: cubría principalmente arrendamientos y Código Civil, el chunking no siempre respetaba los límites de cada artículo y los metadatos eran incompletos. Por ello, se rediseñó el pipeline completo del corpus y se construyó el corpus v2, en junio de 2026, con 13 documentos, 2.786 chunks y un sistema de evaluación cuantitativa.

En paralelo, se realizó una revisión de seguridad que llevó a implementar cabeceras HTTP, protección frente a prompt injection y validación estricta mediante

Pydantic. También se añadió la detección de relaciones entre cláusulas mediante un grafo y se completaron los tests unitarios y de integración. De esta forma, ContratIA evolucionó desde una herramienta inicial de análisis contractual hasta una aplicación funcional compuesta por tres módulos principales: análisis de contratos, chatbot jurídico y generación de contratos.

C.2. Decisiones técnicas que cambiarían en retrospectiva

El desarrollo iterativo permite identificar, al final del proyecto, qué decisiones técnicas podrían haberse abordado de otra manera. En el caso de ContratIA, hay tres aspectos que, con la perspectiva actual, se plantearían de forma diferente.

1. Diseño inicial más extensible del esquema de base de datos

El esquema de base de datos evolucionó de forma orgánica a lo largo del proyecto, lo que obligó a realizar varias migraciones durante el desarrollo. Aunque db_initializer.py gestiona estas migraciones de manera versionada, habría sido conveniente dedicar más tiempo al diseño inicial del modelo de datos, previendo desde fases tempranas entidades como clause_relations y generated_contracts.

No se trata de haber cerrado por completo el esquema desde el inicio, ya que el alcance del proyecto también evolucionó, sino de haber definido una estructura inicial más extensible. Para proyectos futuros, esta experiencia muestra la importancia de anticipar mejor las necesidades de persistencia antes de desarrollar parte de la lógica de aplicación.

2. Incorporación temprana de una estrategia de testing

Los tests se escribieron principalmente en la fase final del proyecto. Aunque finalmente se alcanzó un nivel de cobertura adecuado, haber incorporado una estrategia de testing desde las primeras iteraciones habría permitido detectar antes algunos errores en la lógica de análisis, especialmente en casos como el manejo de cláusulas sin un número de artículo claro.

Además, una base de tests más temprana habría dado más confianza durante las refactorizaciones y ampliaciones del sistema. En futuras versiones, podría plantearse una aproximación más cercana al Test-Driven Development en los componentes críticos, como el análisis de contratos, la validación de respuestas del modelo y la gestión del corpus legal.

3. Separación más clara entre la capa de datos y la capa de negocio

Algunas funciones de los endpoints de FastAPI mezclan lógica de acceso a datos con lógica de negocio. Esta decisión permitió avanzar con rapidez durante las primeras fases, pero a medida que el proyecto creció hizo que ciertas partes del código fueran más difíciles de probar y mantener.

Una separación más clara mediante una capa de repositorio, siguiendo el patrón Repository, habría facilitado el testing, al permitir usar mocks de repositorio en lugar de mocks de la base de datos completa. También habría mejorado la organización del código y reducido el acoplamiento entre la API, la persistencia y la lógica de negocio. Esta separación queda identificada como una deuda técnica razonable para una versión futura.